Desenvolvimento de Aplicações Móveis
Mobile Application Development
DAM

Tutorial 3 - JP Compose

Ana Duarte Correia
ana.correia@isel.pt

António Teófilo
antonio.teofilo@isel.pt

Pedro Fazenda
pedro.fazenda@isel.pt

Abstract

Annotations are widely used in many Android libraries to simplify code and enhance functionality. In this tutorial, you will learn how to create your own Kotlin annotation processor. Following that, you'll explore the MVVM architecture using Jetpack Compose by building a new version of the WeatherApp.

Keywords: annotations - MVVM - Jetpack Compose

Deadline: April 26th, 2026

Annotations, MVVM and Jetpack Compose

Monday, May 3rd, 2025

Contents

List of Figures

1 Tutorial - Annotation Processor [AC NO, AI NO]

In this section, you will explore how annotations work in Kotlin and how to use annotation processing to generate code at compile time. Annotation processing is a powerful tool that allows you to automate repetitive tasks and build more declarative and maintainable applications.

Global note on the use of AI tools. Each task is explicitly labelled with `AC` (Autocomplete) and `AI` (Artificial Intelligence assistance), both marked as `YES` or `NO`. These labels define whether the use of such features is permitted. Compliance with these indications is mandatory.

1.1 Assignment Overview

As a hands-on exercise, you will implement a custom annotation processor in Kotlin that generates a wrapper method based on an annotation parameter. Specifically, you will define a `@Greeting` annotation that accepts a custom message. When a method is annotated with `@Greeting`, your processor will generate a wrapper class that prints the greeting message before calling the original method.
This assignment will be implemented as a multi-module project in IntelliJ IDEA, structured as follows:

- an **annotations** module to define the `@Greeting` annotation;
- a **processor** module to implement the annotation processor and generate the corresponding code;
- an **app** module to test and demonstrate the use of the generated functionality.

1.2 Learning Objectives

By completing this assignment, you will:

- understand how compile-time annotation processing works in Kotlin;
- learn how to set up and organize a multi-module Kotlin project;
- automate method generation using a custom annotation processor;
- gain experience with KotlinPoet, or similar library, for code generation.

Follow the step-by-step instructions provided to build your processor, test its functionality, and see annotation processing in action.

1.3 Step 1: Create the Kotlin JVM Project

Open IntelliJ IDEA and create a new Kotlin JVM Project. Name your project (e.g. `GreetingProcessorProject`)[1].

Select **Gradle** as your build system and complete the project setup. Then, create the following three modules—**annotations**, **processor**, and **app**—as described below:

1. Right-click on the project root.
2. Click New → Module.
3. Select Kotlin (with JVM).
4. Name it `annotations`.
5. Click Finish.

Do the same for the `app` and the `processor` modules.

Set Up Gradle Dependencies for Each Module

Each module needs specific dependencies.

Modify `annotations/build.gradle.kts`

```
plugins {
    kotlin("jvm")
}

group = "org.example"
version = "1.0-SNAPSHOT"

repositories {
    mavenCentral()
}

dependencies {
    testImplementation(kotlin("test"))
    implementation(kotlin("stdlib"))
}

tasks.test {
    useJUnitPlatform()
}
kotlin {
    jvmToolchain(23)
}
```

[1]Do not add sample code

Modify **processor/build.gradle.kts**

This module will contain the annotation processor.

```
plugins {
    kotlin("jvm")
    kotlin("kapt") // Preferred in Kotlin DSL (Gradle KTS)
}

group = "org.example"
version = "1.0-SNAPSHOT"

repositories {
    mavenCentral()
}

dependencies {
    testImplementation(kotlin("test"))
    implementation(kotlin("stdlib"))

    // AutoService to register the processor automatically
    implementation("com.google.auto.service:auto-service:1.1.1")

    // Required for registering the processor
    kapt("com.google.auto.service:auto-service:1.1.1")

    // KotlinPoet for generating Kotlin code
    implementation("com.squareup:kotlinpoet:1.14.2")

    // Include the annotations module
    implementation(project(":annotations"))
}

kapt {
    correctErrorTypes = true
}

tasks.test {
    useJUnitPlatform()
}
kotlin {
    jvmToolchain(23)
}
```

Modify **app/build.gradle.kts**

This module will use the annotation processor.

```
plugins {
    kotlin("jvm")
    kotlin("kapt") // Needed for annotation processing
}

group = "org.example"
version = "1.0-SNAPSHOT"

repositories {
    mavenCentral()
}
```

```
dependencies {
    testImplementation(kotlin("test"))
    implementation(kotlin("stdlib"))

    // Include the annotations module
    implementation(project(":annotations"))

    // Use the annotation processor
    kapt(project(":processor"))
}
tasks.test {
    useJUnitPlatform()
}
kotlin {
    jvmToolchain(23)
}
```

Enable Annotation Processing in IntelliJ

1. Go to File → Settings → Build, Execution, Deployment → Compiler → Annotation Processors.
2. Check "Enable annotation processing".
3. Click OK.

In your **GreetingsProcessorProject**, add this entry in the `gradle.properties` file:

```
kapt.include.compile.classpath=false
```

Verify if your settings.gradle.kts file includes the following:

```
rootProject.name = "GreetingProcessorProject"
include("annotations")
include("processor")
include("app")
```

1.4 Step 2: Create the Annotation Module/Class

The annotation will allow you to specify a greeting message that will later be used by the annotation processor to generate a greeting method.

1. Open the `annotations` Module in IntelliJ

Navigate to the `annotations` module and inside `src/main/kotlin`, create a new package:

```
annotations
```

2. Define the Annotation

Inside the package `com.example.annotations` create a new Kotlin file named `Greeting.kt` and add the following code:

```
package annotations

@Target(AnnotationTarget.FUNCTION)
@Retention(AnnotationRetention.SOURCE)
annotation class Greeting(val message: String)
```

3. Explanation of the Annotation

- `@Target(AnnotationTarget.FUNCTION)`: This annotation can only be applied to functions.
- `@Retention(AnnotationRetention.SOURCE)`: The annotation will not be present at runtime; it is only used at compile time for processing.
- `val message: String`: This parameter allows passing a greeting message as an argument to the annotation. Example Usage: If you apply this annotation to a class:

```
@Greeting("Hello, Kotlin!")
class MyClass
```

4. Sync Gradle

To ensure that the annotation module is correctly recognized, synchronize the Gradle project by clicking `"Load Gradle Changes"` in IntelliJ.

Next Steps

With the annotation defined, the next step is to implement the annotation processor, which will process the annotation and generate the corresponding code.

1.5 Step 3: Create the Annotation Processor Module/Class

Inside the processor project, create a new Kotlin file: GreetingProcessor.kt:

```
package processor
import annotations.Greeting
import com.google.auto.service.AutoService
import com.squareup.kotlinpoet.*
import java.io.File
import javax.annotation.processing.*
import javax.lang.model.SourceVersion
import javax.lang.model.element.ExecutableElement
import javax.lang.model.element.TypeElement
import javax.tools.Diagnostic

@AutoService(Processor::class)
@SupportedSourceVersion(SourceVersion.RELEASE_23)
@SupportedAnnotationTypes("annotations.Greeting")
class GreetingProcessor : AbstractProcessor() {

    override fun process(annotations: MutableSet<out TypeElement>,
        roundEnv: RoundEnvironment): Boolean {
        val classMethodMap = mutableMapOf<TypeElement,
            MutableList<ExecutableElement>>()
```

```
        // Find all methods annotated with @Greeting
        for (element in
            roundEnv.getElementsAnnotatedWith(Greeting::class.java)) {
            if (element is ExecutableElement) {
                val enclosingClass = element.enclosingElement as
                    TypeElement
                classMethodMap.computeIfAbsent(enclosingClass) {
                    mutableListOf() }.add(element)
            }
        }

        // Generate wrapper classes for each class containing annotated
            methods
        for ((classElement, methods) in classMethodMap) {
            generateKotlinWrapperClass(classElement, methods)
        }

        return true
    }

    private fun generateKotlinWrapperClass(classElement: TypeElement,
        methods: List<ExecutableElement>) {
        val packageName =
            processingEnv.elementUtils.getPackageOf(classElement).toString()
        val originalClassName = classElement.simpleName.toString()
        val wrapperClassName = "${originalClassName}Wrapper"

        // Create the wrapper class using composition
        val classBuilder = TypeSpec.classBuilder(wrapperClassName)
            .primaryConstructor(
                FunSpec.constructorBuilder()
                    .addParameter("original", ClassName(packageName,
                        originalClassName))
                    .build()
            )
            .addProperty(
                PropertySpec.builder("original", ClassName(packageName,
                    originalClassName))
                    .initializer("original")
                    .build()
            )
            .addModifiers(KModifier.PUBLIC, KModifier.FINAL)

        // Generate wrapper methods
        for (method in methods) {
            val methodName = method.simpleName.toString()
            val parameters = method.parameters.map { param ->
                ParameterSpec.builder(param.simpleName.toString(),
                    param.asType().asTypeName()).build()
            }
            val arguments = method.parameters.joinToString(", ") {
                it.simpleName.toString() }
            val greetingMessage =
                method.getAnnotation(Greeting::class.java)?.message ?:
                "Hello!"

            val methodBuilder = FunSpec.builder(methodName)
                .addModifiers(KModifier.PUBLIC, KModifier.FINAL)
                .addParameters(parameters)
```

```
                .addStatement("println(%S)", greetingMessage) // Print
                    greeting message
                .addStatement("original.$methodName($arguments)")
                    //Correctly call the original method

            classBuilder.addFunction(methodBuilder.build())
        }

        // Build the Kotlin file
        val file = FileSpec.builder(packageName, wrapperClassName)
            .addType(classBuilder.build())
            .build()

        // Write the generated file
        try {
            val kaptKotlinGeneratedDir =
                processingEnv.options["kapt.kotlin.generated"]
            if (kaptKotlinGeneratedDir != null) {
                file.writeTo(File(kaptKotlinGeneratedDir)) // Correct way
                    to write Kotlin files
            } else {
                processingEnv.messager.printMessage(Diagnostic.Kind.ERROR,
                    "kapt.kotlin.generated not found")
            }
        } catch (e: Exception) {
            processingEnv.messager.printMessage(Diagnostic.Kind.ERROR,
                "Error generating Kotlin file: ${e.message}")
        }
    }
}
```

How it works

1. The processor scans for all methods annotated with `@Greeting`.
2. For each class containing annotated methods, it creates a corresponding wrapper class.
3. The wrapper class uses composition to hold a reference to the original class.
4. For every annotated method, it generates a wrapper method that:
    - Prints the greeting message specified in the annotation; and
    - Delegates the call to the original method with the same parameters.
5. The generated code is written to the build directory using `KotlinPoet`.

1.6 Step 4: Exploring the result of code generation

Let us create a class annotated with `@Greeting`. Add the following `MyClass.kt` file to the `app` module:

```
package com.example.app
import com.example.annotations.Greeting

open class MyClass {
```

```
    @Greeting("Hello from MyClass!")
    open fun sayHello() {
        println("Executing sayHello method")
    }

    @Greeting("Welcome to the compute function!")
    open fun compute() {
        println("Computing something important...")
    }
}
```

After compiling the app package, the annotation processor will generate the following class:

```
<project>/app/build/generated/source/kaptKotlin/<your-package>/MyClassWrapper.kt
```

MyClassWrapper.kt (Auto-Generated)

```
package app

public final class MyClassWrapper(
  public val original: MyClass,
) {
  public final fun sayHello() {
    println("Hello from MyClass!")
    original.sayHello()
  }

  public final fun compute() {
    println("Welcome to the compute function!")
    original.compute()
  }
}
```

1.7 Step 5: Using the Generated Class

Create the Main.kt file in the app module:

```
package com.example.app
import com.example.generated.MyClassWrapper

fun main() {
    val myClass = MyClass()
    val wrappedMyClass = MyClassWrapper(myClass) // Use the wrapper class

    wrappedMyClass.sayHello()
    wrappedMyClass.compute()
}
```

Compile and execute. The expected console output should be the following:

```
Hello from MyClass!
Executing sayHello method
Welcome to the compute function!
Computing something important...
```

The wrapper class displays the greeting message before invoking the original method. If you see this output, congratulations! You have successfully implemented your first annotation processor.

2 Regex Annotation Processor [10%] [AC NO, AI NO]

After successfully completing the previous assignment, you should be ready to tackle another challenge. To earn **10%** from the **30%** assigned to optional challenges, you must implement a new annotation processor named **RegexProcessor** that will handle regular expressions.
The core idea is as follows: you will have a DataProcessor class that takes an input string via its primary constructor. This class includes several abstract methods, each annotated with a regular expression. These methods are responsible for extracting specific parts of the input string that match the provided regular expression. For example:

```
package com.dam
import annotations.Extract

abstract class DataProcessor(val input: String) {
    @Extract(regex = "Name: (\\w+)")
    abstract fun getName(): String?

    @Extract(regex = "Address: (.+)")
    abstract fun getAddress(): String?
}
```

This class will be processed by your annotation processor, which should generate the following class:

```
package com.dam
import kotlin.String
import kotlin.text.Regex

public class DataProcessorExtractor(
  input: String,
) : DataProcessor(input) {

  override fun getName(): String? {
    val match = Regex("Name: (\\w+)").find(input)
    return match?.groupValues?.get(1)
  }

  override fun getAddress(): String? {
    val match = Regex("Address: (.+)").find(input)
    return match?.groupValues?.get(1)
  }
}
```

You can then use the generated class in your Main.kt file as this:

```
package com.dam
import com.dam.DataProcessorExtractor

fun main() {
    val input = "Name: John Address: 123 Street"

    // Using the generated DataProcessorExtractor
    val extractor = DataProcessorExtractor(input)

    println("Name: ${extractor.getName()}")
```

```
    println("Address: ${extractor.getAddress()}")
}
```

With the following expected output:

```
Name: John
Address: 123 Street
```

Following the same structure used in Section 1, create the necessary packages, define the annotation, and implement the annotation processor. Compile the project and verify that the generated output matches the expected result.

3 Android - MVVM and Jetpack Compose [AC YES, AI NO]

Before starting this section, we recommend that students read and follow the **View-Model and State in Compose** codelab, created by the Google Developers Training team:

```
https://developer.android.com/codelabs/basic-android-kotlin-compose-viewmodel-and-state
```

This codelab provides a step-by-step guide to understanding the **ViewModel**, one of the core architecture components in the Android Jetpack Compose libraries, designed to help manage and retain UI-related data. We strongly recommend you to follow the codelab and build the **Unscramble** app in a step by step way. However, if you prefer, you can download the final solution (available at the end of the codelab) and review the source code, to grasp the key concepts.
In this assignment, you will rebuild the *WeatherApp* developed in the previous tutorial, adapting it to follow the **MVVM** (**Model–View–ViewModel**) architecture, outlined in Figure 1 and in the codelab above, using *Jetpack Compose* instead of XML layouts. Therefore, like in the **Unscramble** app, your project should be organized into three separate packages:

1. **data** — contains all data layer files;
2. **viewmodel** — contains the viewmodel of the application; and
3. **ui** — contains all files related to the user interface.

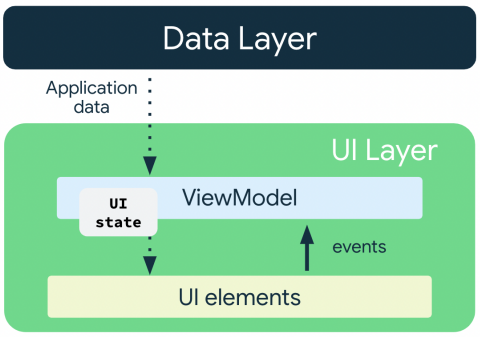

Figure 1: MVVM architecture.

3.1 The data layer

In the **data** package, include the `WeatherData.kt` file from your previous assignment, and add the following **ktor**-based `WeatherAPIClient`:

Listing 1: WeatherApiClient.kt

```
package com.example.cooljetpackweatherapp.data
import ...

object WeatherApiClient {
  private val client = HttpClient {
    install(ContentNegotiation) {
        json(Json {
            prettyPrint = true
            isLenient = true
            ignoreUnknownKeys = true
        }) // Ignores extra JSON fields
    }
  }

  suspend fun getWeather(lat: Float, lon: Float): WeatherData? {
    val reqString = buildString {
      append("https://api.open-meteo.com/v1/forecast?")
      append("latitude=${lat}&longitude=${lon}&")
      append("current_weather=true&")
      append("hourly=temperature_2m,weathercode,pressure_msl,windspeed_10m")
    }
    System.out.println("Getting URL: $reqString")
    return try {
        client.get(reqString).body() // Parses JSON into WeatherData
    } catch (e: Exception) {
        e.printStackTrace()
        null
    }
  }
}
```

In order for this client to function correctly, your `WeatherData` data classes must be annotated with the `@Serializable` annotation.
In the example provided in this assignment, the application displays only the following weather parameters:

- `temperature`;
- `windSpeed`;
- `windDirection`;
- `weathercode`; and
- `seaLevelPressure`.

However, you are encouraged to include additional parameters and adapt the application accordingly.

3.2 The User Interface (UI) [5%]

In this project, the user interface will be built using *Jetpack Compose*, with all composable functions placed in files organized under the **ui** package. This ensures a clear separation of concerns and aligns with the MVVM architecture.

In your `MainActivity`, the `setContent` block should call a top-level composable function — for example, `WeatherUI` — which is responsible for creating or retrieving the appropriate `ViewModel` and rendering the UI based on the current application state.
If you are unsure how to structure this, revisit the **Unscramble** app example for reference. Pay special attention to how composables are organized and how state flows from the ViewModel to the UI through state hoisting and observable properties.
The `WeatherUI` composable should be placed in the `WeatherScreen.kt` file within the **ui** package, following a structure similar to the one below:

Listing 2: WeatherScreen.kt

```
package com.example.cooljetpackweatherapp.ui
import ...

@Composable
fun WeatherUI(weatherViewModel: WeatherViewModel = viewModel()) {

    val weatherUIState by weatherViewModel.uiState.collectAsState()
    val latitude = weatherUIState.latitude
    val longitude = weatherUIState.longitude
    val temperature = weatherUIState.temperature
    val windSpeed = weatherUIState.windspeed
    val windDirection = weatherUIState.winddirection
    val weathercode = weatherUIState.weathercode
    val seaLevelPressure = weatherUIState.seaLevelPressure
    val time = weatherUIState.time

    val configuration = LocalConfiguration.current

    val day = true    // Must change this in the future
    val mapt = getWeatherCodeMap();
    val wCode = mapt.get(weathercode)
    val wImage = when(wCode) {
        WMO_WeatherCode.CLEAR_SKY,
        WMO_WeatherCode.MAINLY_CLEAR,
        WMO_WeatherCode.PARTLY_CLOUDY -> if (day) wCode?.image + "day"
            else wCode?.image + "night"
        else -> wCode?.image
    }
    val context = LocalContext.current
    val wIcon = context.resources.getIdentifier(wImage, "drawable",
        context.packageName)

    if (configuration.orientation == Configuration.ORIENTATION_LANDSCAPE) {
        LandscapeWeatherUI(
            wIcon,
            latitude,
            longitude,
            temperature,
            windSpeed,
            windDirection,
            weathercode,
            seaLevelPressure,
            time,
            onLatitudeChange = {
                newValue -> newValue.toFloatOrNull()?.let {
                    weatherViewModel.updateLatitude(it) }
            },
            onLongitudeChange = {
```

```
                newValue -> newValue.toFloatOrNull()?.let {
                    weatherViewModel.updateLongitude(it) }
            },
            onUpdateButtonClick = {
                weatherViewModel.fetchWeather()
            }
        )
    } else {
        PortraitWeatherUI(
            wIcon,
            latitude,
            longitude,
            temperature,
            windSpeed,
            windDirection,
            weathercode,
            seaLevelPressure,
            time,
            onLatitudeChange = {
                newValue ->
                newValue.toFloatOrNull()?.let {
                    weatherViewModel.updateLatitude(it) }
            },
            onLongitudeChange = {
                newValue ->
                newValue.toFloatOrNull()?.let {
                    weatherViewModel.updateLongitude(it) }
            },
            onUpdateButtonClick = {
                weatherViewModel.fetchWeather()
            }
        )
    }
}

@Composable
fun PortraitWeatherUI(
    wIcon: Int,
    latitude: Float,
    longitude: Float,
    temperature: Float,
    windSpeed: Float,
    windDirection: Int,
    weathercode: Int,
    seaLevelPressure: Float,
    time: String,
    onLatitudeChange: (String) -> Unit,
    onLongitudeChange: (String) -> Unit,
    onUpdateButtonClick: () -> Unit,
) {
    // ToDo
}

@Composable
fun LandscapeWeatherUI(
    wIcon: Int,
    latitude: Float,
    longitude: Float,
    temperature: Float,
    windSpeed: Float,
```

```
    windDirection: Int,
    weathercode: Int,
    seaLevelPressure: Float,
    time: String,
    onLatitudeChange: (String) -> Unit,
    onLongitudeChange: (String) -> Unit,
    onUpdateButtonClick: () -> Unit,
) {
    // ToDo
}
```

Figures 2 and 3 present sample layouts for portrait and landscape orientations. These should be regarded as illustrative examples - you are encouraged to be creative and design your own user interface. **5%** from the **30%** allocated for an innovative design of the interface (UI).

There are several resources available to help you learn Jetpack Compose. For example, you can explore Stevdza-San's YouTube channel:

`https://youtu.be/ERBEWmfz6h0?si=_lQ2tL9NEzEsHL6d`

You may also find the following links useful:

- **Compose Tutorial:** `https://developer.android.com/develop/ui/compose/tutorial`
- **Material Design:** `https://m3.material.io/`

The current user interface is organized using several `Compose` functions:

- `CoordinatesCard`;
- `WeatherCard`; and
- `WeatherRow`.

Each function is defined in its own file within the `ui` package. These components are reused in both portrait and landscape modes, to display an Image [2], representing the weather status (associated with the weather code), the coordinates, the weather information card, and the individual rows within the weather display.
However, you are free to structure your UI as you see fit. Just keep in mind that each composable should follow the principle of single responsibility and that UI logic must remain separated from business logic. Attempt to keep composables clean, modular, and reusable.

3.3 ViewModel Responsibilities and Project Architecture

The `ViewModel` is responsible for maintaining the current state of the user interface. It handles events such as the update button press, which triggers a weather data fetch

[2] Use the same image resources provided in the previous assignment.

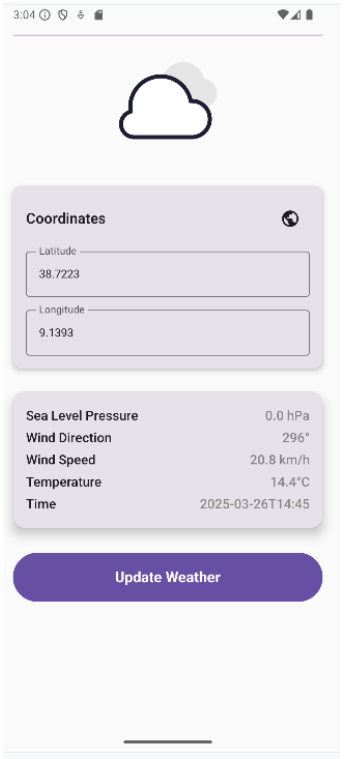

Figure 2: Cool Jetpack Weather App main screen (portrait).

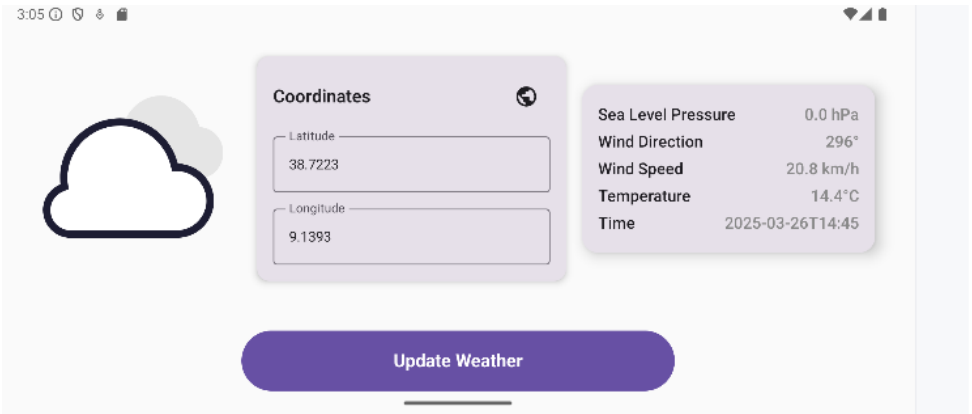

Figure 3: Cool Jetpack Weather App main screen (landscape).

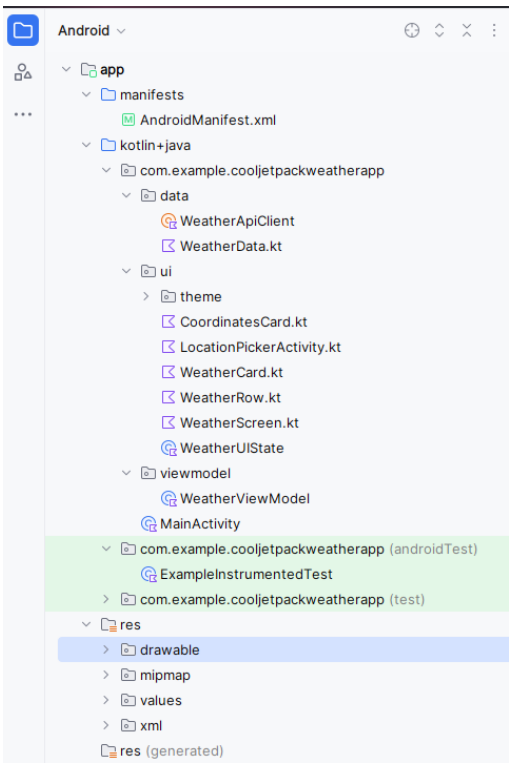

Figure 4: The Cool Jetpack Weather App project structure.

and subsequently updates the UI state. In addition, it processes latitude and longitude updates sent from the UI components.
This separation ensures a unidirectional data flow and promotes a clean architecture, where the UI observes changes to the state managed by the `ViewModel`. This approach improves the testability and maintainability of the application.
The overall project structure of the *Cool Jetpack Weather App* is illustrated in Figure 4.

3.4 Multilanguage Support

To support internationalization, your application must include multilanguage support, with at least English and Portuguese (or another language) as available languages. All user-facing strings must be defined in the `res > values > strings.xml` file, and the corresponding translations must be provided in `res > values-pt > strings.xml`. Hard-coded strings in the UI are not allowed. This approach ensures that your app is accessible to a broader audience and adheres to best practices in Android development.

3.5 Optional Features [5%]

To earn **5%** from the **30%** assigned to optional challenges, you must add a **location picker** and the support of **favorite named locations**.

Location picker

For the location picker, in your `CoordinatesCard`, add a small icon that, when tapped, launches a `LocationPickerActivity`. This activity should display a world map, allow

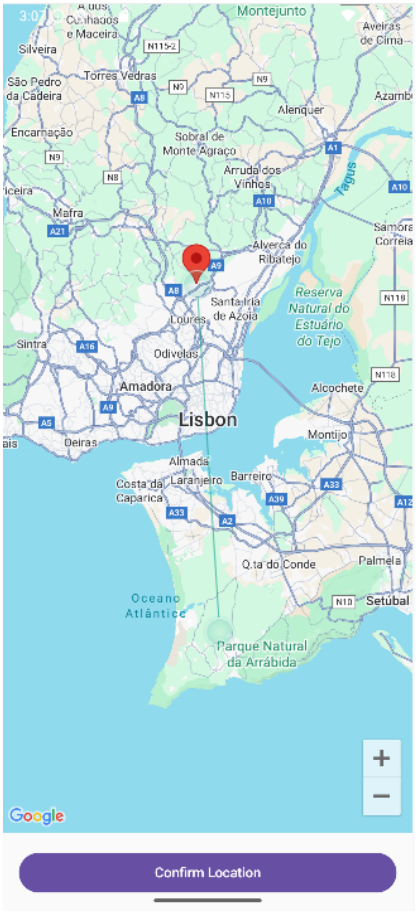

Figure 5: The Location Picker Activity.

the user to select a location, and return the selected coordinates back to the `CoordinatesCard`. Figure 5 shows an example of how this activity might look.
You can use the `Icons.Default.Public` icon from Jetpack Compose, which corresponds to the "Public" icon in Google's Material Icons set. This icon visually represents a globe or planet Earth and is commonly used to indicate:

- Global access;
- Public visibility;
- World or internet-related features; and
- Geographic or map-related actions.

There are several ways to incorporate map features into the `LocationPickerActivity`. The most straightforward approach is to use Google Maps services. To do this, you will need to:

1. Create an account on Google Cloud Platform;
2. Set up a new project within your Google Cloud account;
3. Enable the Google Maps SDK for Android in your project [3]; and

[3]Under APIs and Services

4. Generate an API key to access Google Maps services.

Once configured, you can integrate the map into your activity and implement the necessary logic to return the selected coordinates to the main application.
Don't forget to add your API key and register your new activity in the `AndroidManifest.xml` file.

Listing 3: AndroidManifest.xml

```
...
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.CoolJetpackWeatherApp"
        android:usesCleartextTraffic="true"
        tools:targetApi="31">
        <activity android:name=
           "com.example.cooljetpackweatherapp.ui.LocationPickerActivity" />
        <meta-data
            android:name="com.google.android.geo.API_KEY"
            android:value="XXXXXX-----YYYYYY" />

...
```

Favorite named locations

For **favorite named locations**, you should allow to register the coordinates associated with a location name. You should also display the names of the recorded locations in a horizontally scrollable list. Finally, when a user taps on one of these items, the location's name and coordinates should be set to active and its weather information should be displayed.

4 Assisted code generation – MIP-3 [10%] [AC YES, AI YES]

Your Mission Impossible Possible – MIP-3

Plan & execute with AI-Assisted Android App Development with AntiGravity.

This MIP is still in preparation.

It will be available soon.

You're All (almost) Set — Happy Coding!